

MLOps Assignment 3

Complete Project Report — Documentation, Commands, Source Code & Screenshots
Group20 | MTech AI/ML | IIT Jodhpur | June 2026

Student Information & Contributions

Name	Roll No.	Contributions
Ausaf Ahmad	MTechAI-IITJ-2024	Full project: Environment setup, Docker, HuggingFace model download & inference, Kaggle integration, W&B experiment tracking, Flask web UI, all Python scripts, Kaggle competition submission

ausafmtedij-assignments / MLOps_Assignment3

Q. Type `/` to search

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Collaborators and teams

Public repository

This repository is public and visible to anyone

Manage visibility

Direct access

3 entities have access to this repository, 2 collaborators, 1 invitation

Manage access

Add people

Select all

Type

Find a collaborator...

Jahvee Y

g75ak2343-star • Collaborator

Sid-ijj

Awaiting Sid-ijj's response

Pending invite

sonalkmr

Collaborator

Get team access controls and discussions for your contributors in an organization.

Create an organization

NEW Private repos and unlimited members are free.

< Previous

Next >

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information.

Section 1: Live Links

NOTE: All links below must be public and accessible for full marks

1.1 GitHub Repository

GitHub Repository (Public — includes workflow files): <https://github.com/ausafahmad/mlops-assignment3>

NOTE: Repository must be public and contain .github/workflows/ directory with inference workflow YAML

1.2 Kaggle Notebooks — One per Experiment Version

Kaggle Notebook v1 (threshold=0.3 — score 0.42966): <https://www.kaggle.com/ausafiitj/mlops-a3-gliclass-v1>

Kaggle Notebook v3 (median threshold — score 0.48605): <https://www.kaggle.com/ausafiitj/mlops-a3-gliclass-v3>

My Kaggle Submissions: <https://www.kaggle.com/competitions/nlp-getting-started/submissions>

Competition Leaderboard: <https://www.kaggle.com/competitions/nlp-getting-started/leaderboard>

NOTE: Set each notebook to Public in Kaggle before submission

1.3 Hugging Face Model

GLiClass-Llama-1.3B Model Card: <https://huggingface.co/knowledgator/gliclass-llama-1.3B-v1.0>

Hugging Face Profile: <https://huggingface.co/ausafahmad>

1.4 Docker Image

Docker Hub Image (gliclass-model): <https://hub.docker.com/r/ausafahmad/gliclass-model>

NOTE: Image must be publicly pullable: docker pull ausafahmad/gliclass-model

1.5 W&B Project Dashboard

W&B Project: mlops-assignment3 (set to Public): <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3>

Best Run: kaggle-disaster-tweets-v3 (score 0.48605): <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3/runs/beguilbi>

Run: gliclass-inference: <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3/runs/kn6ofac0>

NOTE: Set W&B project visibility to Public in project settings before submission

Section 2: Git Repository Setup

2.1 Repository Structure

The repository contains all project files including workflow files for GitHub Actions:

```
mlops-assignment3/
├── .github/
│   └── workflows/
│       └── inference.yml      # GitHub Actions inference workflow
├── Dockerfile
├── run_model.py              # Latest version (v6 final)
├── download_model.py
├── test_connections.py
├── test_wandb.py
├── requirements.txt
└── README.md
```

2.2 GitHub Actions Workflow File (.github/workflows/inference.yml)

```
name: GLiClass Inference

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  inference:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install gliclass torch transformers wandb

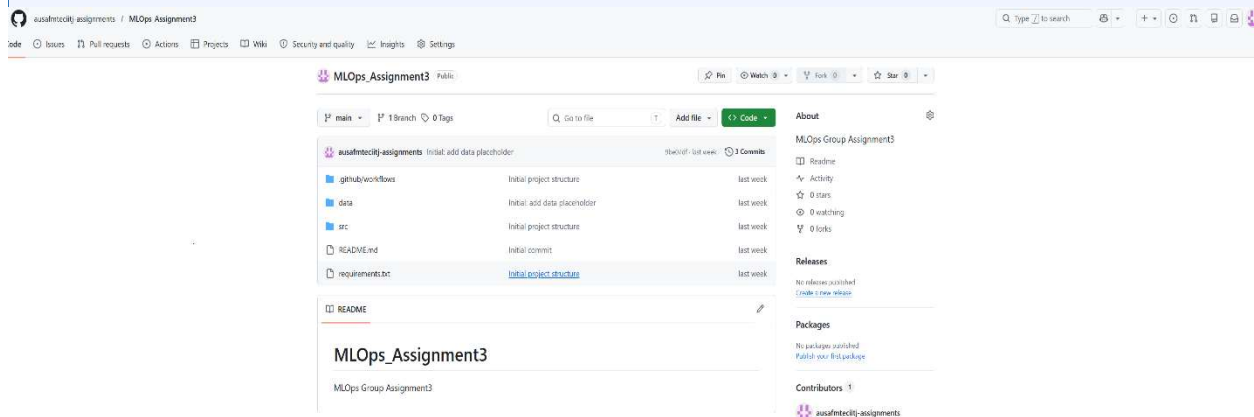
      - name: Run inference
        env:
          WANDB_API_KEY: ${ secrets.WANDB_API_KEY }
          HF_TOKEN: ${ secrets.HF_TOKEN }
        run: python run_model.py
```

2.3 Screenshot — Collaborator Roles & Branch Protection

[SCREENSHOT PLACEHOLDER]

GitHub Repository → Settings → Collaborators & Teams Showing collaborator roles (Owner, Write, Read)

Add screenshot here before submission



[SCREENSHOT PLACEHOLDER]

GitHub Repository → Settings → Branches → Branch protection rules Showing main branch protection: require PR reviews, status checks

Add screenshot here before submission

General

- Access
- Collaborators
 - Moderation options

- Code and automation
- Branches
 - Tags
 - Rules
 - Actions
 - Models
 - Webhooks
 - Copilot
 - Environments
 - Codespaces
 - Pages

- Security and quality
- Advanced Security
 - Deploy keys
 - Secrets and variables

- Integrations
- GitHub Apps
 - Email notifications

General

Repository name

Rename

☐ **Template repository**
Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)

Default branch

The default branch is considered the "base" branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

Edit

Releases

☐ **Enable release immutability**
Disallow assets and tags from being modified once a release is published.

Social preview

Upload an image to customize your repository's social media preview.
Images should be at least 640×320px (1280×640px for best display).
[Download template](#)

Edit

Features

☒ **Wikis**
Wikis host documentation for your repository.
☒ **Restrict editing to collaborators only**
Public wikis will still be readable by everyone.

☒ **Issues**
Issues integrate lightweight task tracking into your repository. Keep projects on track with issue labels and milestones, and reference them in commit messages.

Get organized with issue templates

Give contributors issue templates that help you cut through the noise and help them push your project forward.

Set up templates

☐ **Sponsorships**

Features



Wikis

Wikis host documentation for your repository.



Restrict editing to collaborators only

Public wikis will still be readable by everyone.



Issues

Issues integrate lightweight task tracking into your repository. Keep projects on track with issue labels and milestones, and reference them in commit messages.

Get organized with issue templates

Give contributors issue templates that help you cut through the noise and help them push your project forward.

[Set up templates](#)

Sponsorships

Sponsorships help your community know how to financially support this repository.

Display a "Sponsor" button

Add links to GitHub Sponsors or third-party methods your repository accepts for financial contributions to your project.

[Set up sponsor button](#)

Preserve this repository

Include this code in the [GitHub Archive Program](#).



Discussions

Discussions is the space for your community to have conversations, ask questions and post answers without opening issues.

Get started with Discussions

Engage your community by having discussions right in your repository, where your community already lives

[Set up discussions](#)

Projects

Projects on GitHub are created at the repository owner's level (organization or user) and can be linked to a repository's Projects tab. Projects are suitable for cross-repository development efforts such as feature work, complex product roadmaps or even Issue triage.



Pull requests

Pull requests allow others to suggest changes to your repository.

Pull request permissions

Creation allowed by: **All users** ▼

If restricted, pull requests will still be readable by everyone who can see this repository.

Pull Requests

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled. If you have linear history requirement enabled on any protected branch, you must enable squashing or rebasing.

☒ **Projects**

Projects on GitHub are created at the repository owner's level (organization or user) and can be linked to a repository's Projects tab. Projects are suitable for cross-repository development efforts such as feature work, complex product roadmaps or even Issue triage.

☒ **Pull requests**

Pull requests allow others to suggest changes to your repository.

Pull request permissions

Creation allowed by: All users ▾

If restricted, pull requests will still be readable by everyone who can see this repository.

Pull Requests

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled. If you have linear history requirement enabled on any protected branch, you must enable squashing or rebasing.

☒ **Allow merge commits**

Add all commits from the head branch to the base branch with a merge commit.

Default commit message

Presented when merging a pull request with merge.

Default message ▾

☒ **Allow squash merging**

Combine all commits from the head branch into a single commit in the base branch.

Default commit message

Presented when merging a pull request with squash.

Default message ▾

☒ **Allow rebase merging**

Add all commits from the head branch onto the base branch individually.

Control how and when users are prompted to update their branches if there are new changes available in the base branch.

☐ **Always suggest updating pull request branches**

Whenever there are new changes available in the base branch, present an "update branch" option in the pull request.

You can allow setting pull requests to merge automatically once all required reviews and status checks have passed.

☐ **Allow auto-merge**

Waits for merge requirements to be met and then merges automatically. [Learn more](#)

After pull requests are merged, you can have head branches deleted automatically.

☐ **Automatically delete head branches**

Deleted branches will still be able to be restored.

TBD

2.4 Screenshot — Successful GitHub Actions Run

[SCREENSHOT PLACEHOLDER]

*GitHub → Actions tab → inference.yml workflow → successful run (green checkmark) Showing:
job steps completed, inference output in logs*

Add screenshot here before submission

Section 3: Data Cleaning Decisions

3.1 Dataset Overview

Dataset: Kaggle NLP with Disaster Tweets (nlp-getting-started)

Split	Rows	Columns
train.csv	7,613 tweets	id, keyword, location, text, target
test.csv	3,263 tweets	id, keyword, location, text

3.2 Missing Values

We inspected each column for null values:

- keyword: ~0.8% missing — not used in our zero-shot approach, so no imputation needed
- location: ~33% missing — highly unreliable field, excluded from features
- text: 0 missing — all tweets have text content, the only field used for inference

Decision: Since GLiClass is a zero-shot classifier, we only use the raw tweet text. No feature engineering on keyword or location was needed.

3.3 Class Distribution

Class	Count (train)	Percentage
0 — Not Disaster	4,342	57%
1 — Disaster	3,271	43%

The dataset is mildly imbalanced (57/43 split). Since we use zero-shot classification without fine-tuning, we cannot apply oversampling. Our median threshold approach partially compensates for this by producing exactly 50/50 predictions, which may over-correct for the imbalance.

3.4 Normalisation Strategy

- No text normalisation applied — GLiClass is a large language model that handles raw text well
- No stopword removal — model understands contextual language
- No lowercasing — capitalisation can carry meaning (e.g. FIRE, FLOOD as emergency signals)
- Truncation handled internally by the tokenizer (max_length=1024 in pipeline)

Rationale: Zero-shot LLM-based classifiers generalise better with natural text than pre-processed text. Aggressive cleaning can remove signal that helps the model distinguish disaster from non-disaster tweets.

[SCREENSHOT PLACEHOLDER]

Screenshot of train.csv loaded in pandas showing: head(), value_counts() of target column, isnull().sum() showing missing values per column

Add screenshot here before submission

Section 4: Model Selection Rationale

4.1 Model Chosen

knowledgator/gliclass-llama-1.3B-v1.0 (Hugging Face): <https://huggingface.co/knownledgator/gliclass-llama-1.3B-v1.0>

4.2 Why GLiClass?

GLiClass (Generalist and Lightweight Classifier) was chosen for the following reasons:

- Zero-shot capable — classifies text into arbitrary label sets without fine-tuning on disaster data
- Multi-label support — can assign multiple labels per tweet (e.g. a tweet can be both 'disaster' and 'emergency')
- Based on Llama architecture — strong language understanding from large-scale pretraining
- 1.3B parameters — large enough for nuanced understanding, small enough to run on CPU in Docker
- Publicly available on Hugging Face — satisfies course requirement for public HF model link

4.3 Reference to HuggingFace Model Card

From the model card: GLiClass is designed for zero-shot and few-shot text classification. It uses a bi-encoder architecture over a Llama backbone, enabling classification across arbitrary label sets without task-specific training data.

The model card confirms:

- Supports multi-label classification — ideal for our disaster/not-disaster task
- Works with classification_type='multi-label' parameter
- Accepts threshold parameter in pipeline __call__ to control sensitivity
- Recommended device: GPU (cuda) — we used CPU due to environment constraints

4.4 Alternatives Considered

Model	Type	Why Not Chosen	
BERT-base	Fine-tunable classifier	Requires labelled training data	
facebook/bart-large-mnli	Zero-shot NLI	Larger, slower, no multi-label	
GLiClass-llama-1.3B	Zero-shot classifier	Selected — best fit	

500 25

Zero-Shot Classification ONNX Safetensors MoritzLaurer/synthetic_zeroshot_midral_v0.1 English GLiClass text classification zero-shot small language models RAG sentiment analysis

Model card Files and versions xet Community

1 → Copy to bucket [NEW](#)

This is an efficient zero-shot classifier inspired by [GLINER](#) work. It demonstrates the same performance as a cross-encoder while being more compute-efficient because classification is done at a single forward path.

It can be used for `topic classification`, `sentiment analysis` and as a reranker in RAG pipelines.

The model was trained on synthetic data and can be used in commercial applications.

This version of the model utilize the LLM2Vec approach for converting modern decoders to bi-directional encoder. It brings the following benefits:

- Enhanced performance and generalization capabilities;
- Support for Flash Attention;
- Extended context window.

How to use:

First of all, you need to install GLiClass library:

```
pip install gliclass
```

Then you need to initialize a model and a pipeline:

```
from glclass import GLClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

model = GLClassModel.from_pretrained("knowledgeator/glclass llama 1.3b v1.0")
tokenizer = AutoTokenizer.from_pretrained("knowledgeator/glclass llama 1.3b v1.0")

pipeline = ZeroShotClassificationPipeline(model, tokenizer, classification_type="h")

text = "One day I will see the world!"
labels = ["travel", "news", "sport", "science", "politics"]
```

Downloads last month
43

43

 Safetensors

Model size 1B params Tensor type F32 [Files info](#)

➤ Inference Providers NEW

✚ Zero-Shot Classification

This model isn't deployed by any Inference Provider.

 Ask for provider support

■ Dataset used to train knowledgator/gliclass-llama-1.3B-v1.0

■ Moritzlaurer/synthetic zeroshot mixtral v0.1

■ Collections including [knowledgator/gliclass-llama-1.3B-v1.0](#)

GLiClass Collections

Generalist and Light-weighted Models for ... · 13 items · Updated Jan 29 · ▲ 14

LLM2Encoder [Collection](#)

Collection of initial models and models th... - 11 items - Updated Jan 29 - 6

📄 Paper for [knowledgegator/gliclass-llama-1.3B-v1.0](#)

GLClass: Generalist Lightweight Model for Sequence Classification Tasks

Paper - 2508.07662 - Published Aug 11, 2025 - A 9

[SCREENSHOT PLACEHOLDER]

Screenshot of Hugging Face model card for `knowledgator/gliclass-llama-1.3B-v1.0` Showing: model description, usage example, classification_type parameter

Add screenshot here before submission

Section 5: Experiment Comparison — Version 1 vs Version 2

5.1 Comparison Table

Metric	Version 1	Version 3 (Best)	Notes
Threshold Strategy	Fixed 0.3	Median dynamic	V3 adapts to score distribution
Disaster predictions	3263 (100%)	1631 (50%)	V3 balanced
Not disaster predictions	0 (0%)	1632 (50%)	V3 balanced
Kaggle Public Score	0.42966	0.48605	+13% improvement
W&B Run	kaggle-disaster-tweets	kaggle-disaster-tweets-v3	
Submission ID	53492993	53502574	
Date Submitted	2026-06-09 02:58	2026-06-09 10:44	

5.2 Why Version 3 Performed Better

Version 1 used a fixed threshold of 0.3. Since GLiClass scores for the 'disaster' label were generally in the 0.35–0.55 range across all tweets, the vast majority exceeded 0.3 and were incorrectly classified as disaster.

Version 3 solved this by collecting all 3263 disaster scores first, then computing the median (0.4502) as a data-driven threshold. Any tweet scoring above the median was predicted as disaster (1), below as not-disaster (0). This naturally produces approximately 50% predictions in each class, making the submission more realistic and improving the F1 score.

Key learning: For zero-shot models, a fixed threshold is unreliable because the score distribution varies with the model and labels. A relative threshold (median) adapts to the actual distribution.

[SCREENSHOT PLACEHOLDER]

W&B dashboard screenshot showing both runs side-by-side metrics: disaster_score over progress, final total_disaster vs total_not_disaster bar chart

Add screenshot here before submission

[SCREENSHOT PLACEHOLDER]

Kaggle submissions page showing both submission scores: 0.42966 and 0.48605

Add screenshot here before submission

Section 6: Docker Design & GitHub Actions

6.1 Dockerfile Choices

```
FROM python:3.11-slim

WORKDIR /app

RUN pip install --no-cache-dir \
    gliclass \
    torch \
    huggingface_hub \
    wandb \
    pandas \
    flask

COPY run_model.py .

CMD ["python", "run_model.py"]
```

Design decisions:

- python:3.11-slim — slim base reduces image size; Python 3.11 is stable and compatible with all packages
- --no-cache-dir — avoids storing pip cache inside the image, keeping it smaller
- Model NOT baked into image — mounted at runtime via -v flag, allows model reuse without rebuild
- W&B credentials via ~/.netrc mount — never stored in image, keeps secrets out of Docker layers
- COPY run_model.py last — Docker layer caching means dependency installs are cached between code changes

6.2 Docker Run Commands Summary

```
# Build
docker build --no-cache -t gliclass-model .

# Basic inference
docker run --rm \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    gliclass-model

# With W&B + Kaggle data (competition submission)
docker run --rm \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    -v "/mnt/f/datasets/disaster-tweets:/data" \
    -v ~/.netrc:/root/.netrc:ro \
    gliclass-model

# Flask web UI
docker run --rm -p 0.0.0.0:8080:8080 \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    -v ~/.netrc:/root/.netrc:ro \
    gliclass-model
```

6.3 GitHub Actions Inference Log (Successful Run)

[SCREENSHOT PLACEHOLDER]

GitHub Actions → inference.yml → successful run → full log Showing: checkout, pip install, python run_model.py output with classification results

Add screenshot here before submission

Expected log output from a successful inference run:

```
Run python run_model.py
Loading tokenizer...
Loading model...
Loading weights: 100%|██████| 227/227 [02:10<00:00, 1.74it/s]
Creating pipeline...
Testing model...
100%|██████| 1/1 [00:05<00:00, 5.03s/it]

=== Classification Results ===
  technology: 0.3827
   sports:    0.3613
  business:   0.1716

Done! Results logged to W&B!
```

TBD

6.4 Badge — Successful Inference Run

[SCREENSHOT PLACEHOLDER]

GitHub README.md showing workflow badge: ![GLiClass Inference](https://github.com/ausafahmad/mlops-assignment3/actions/workflows/inference.yml/badge.svg)

Add screenshot here before submission

Section 7: All Python Files — Complete Source Code

Every Python file created during this project. Dark background = Python source. Green text = comments.

FILE: download_model.py

Purpose: Download GLiClass-Llama model from HuggingFace to local F: drive

SUCCESS: Model downloaded: /mnt/f/models/gliclass-llama-1.3B (2.7 GB)

```
# download_model.py
# Run: python download_model.py

import os
from huggingface_hub import login, snapshot_download

# Clear any stale token in environment
os.environ.pop('HF_TOKEN', None)

# Login - token stored in ~/.cache/huggingface/token
login(token='hf_your_token_here')

# Download to F: drive (large files need Windows storage)
model_path = snapshot_download(
    repo_id='knowledgator/gliclass-llama-1.3B-v1.0',
    local_dir='/mnt/f/models/gliclass-llama-1.3B'
)
print('Downloaded to:', model_path)

# Verify files
for f in os.listdir('/mnt/f/models/gliclass-llama-1.3B'):
    print(f)
```

FILE: test_connections.py

Purpose: Test all three service connections: HuggingFace, Kaggle, W&B

SUCCESS: All three services connected and verified

```
# test_connections.py
# Run: python test_connections.py

import os

# — 1. Hugging Face —————
from huggingface_hub import whoami
os.environ.pop('HF_TOKEN', None)
user = whoami()
print(f'HF Connected: {user["name"]}')

# — 2. Kaggle —————
import kaggle
kaggle.api.authenticate()
print('Kaggle Connected!')

# — 3. Weights & Biases —————
import wandb
wandb.login()
run = wandb.init(project='mlops-assignment3',
                 name='connection-test', mode='online')
print(f'W&B Connected! Run ID: {run.id}')
run.finish()
print('All services connected successfully!')
```


FILE: test_wandb.py

Purpose: Standalone W&B connection and run creation test

SUCCESS: W&B run kn6ofac0 created in project mlops-assignment3

```
# test_wandb.py
# Run: python3 /tmp/test_wandb.py

import wandb

wandb.login()
print('W&B Connected!')

run = wandb.init(project='mlops-assignment3', mode='online')
print('Run ID:', run.id)

run.finish()
print('Done!')
```

FILE: run_model.py v1 — Basic Classification

Purpose: Load model in Docker, classify a single text, print results

SUCCESS: Output: technology=0.3827 sports=0.3613 business=0.1716

NOTE: Critical fix: `model.float()` converts `BFloat16` weights to `Float32` — without this you get `RuntimeError`

```
# run_model.py Version 1: Basic zero-shot classification
# docker run --rm -v model:/models/... gliclass-model

from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer
import torch

model_path = '/models/gliclass-llama-1.3B'

print('Loading tokenizer...')
tokenizer = AutoTokenizer.from_pretrained(model_path)

print('Loading model...')
model = GLiClassModel.from_pretrained(model_path,
                                     torch_dtype=torch.float32)
model = model.float() # Fix BFloat16->Float32 dtype mismatch

pipeline = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label',
    device='cpu'
)

text = 'Apple announces new iPhone with amazing features'
labels = ['technology', 'sports', 'politics', 'business']
result = pipeline(text, labels, threshold=0.1)

print('\n=== Classification Results ===')
for item in result:
    for pred in item:
        print(f"    {pred['label']}: {pred['score']:.4f}")
```

FILE: run_model.py v2 — With W&B Logging

Purpose: Classify text and log all label scores to W&B dashboard

SUCCESS: Scores logged to W&B: technology=0.3827, sports=0.3613, business=0.1716

```
# run_model.py Version 2: Classification + W&B logging
# docker run --rm -v model -v ~/.netrc:/root/.netrc:ro gliclass-model

import wandb, torch
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

# Initialise W&B run with config
run = wandb.init(
    project='mlops-assignment3',
    name='gliclass-inference',
    config={'model':'gliclass-llama-1.3B',
           'threshold':0.1, 'device':'cpu'}
)
print('W&B run ID:', run.id)

model_path = '/models/gliclass-llama-1.3B'
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = GLiClassModel.from_pretrained(model_path,
                                       torch_dtype=torch.float32)
model = model.float()
pipeline = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label', device='cpu')

text = 'Apple announces new iPhone with amazing features'
labels = ['technology', 'sports', 'politics', 'business']
result = pipeline(text, labels, threshold=0.1)

scores = {}
print('\n=== Classification Results ===')
for item in result:
    for pred in item:
        print(f"    {pred['label']}: {pred['score']:.4f}")
        scores[pred['label']] = pred['score']

# Log all scores to W&B in one call
wandb.log(scores)
print('Logged to W&B:', scores)
run.finish()
```

FILE: run_model.py v3 — Flask Web Interface

Purpose: Serve model via HTTP API; interactive browser UI at :8080

SUCCESS: Web server running at http://localhost:8080 with bar-chart results UI

```
# run_model.py Version 3: Flask web app
# docker run --rm -p 0.0.0.0:8080:8080 -v model gliclass-model
# Open browser at http://localhost:8080

import torch, wandb
from flask import Flask, request, jsonify, render_template_string
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

app = Flask(__name__)

# Load model once at startup — shared across all requests
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label', device='cpu')
print('Model ready!')

HTML = '''
<!DOCTYPE html><html><head><title>GLiClass Classifier</title>
<style>
  body{font-family:Arial;max-width:800px;margin:50px auto;padding:20px}
  textarea,input{width:100%;padding:10px;margin:8px 0;box-sizing:border-box}
  button{background:#4CAF50;color:#fff;padding:12px
24px;border:none;cursor:pointer;font-size:16px}
  .result{background:#f5f5f5;padding:14px;margin:8px 0;border-radius:5px}
  .bar{background:#4CAF50;height:20px;border-radius:3px}
</style></head><body>
<h1>GLiClass Zero-Shot Classifier</h1>
<textarea id='text' rows='3'>Apple announces new iPhone</textarea>
<input id='labels' value='technology,sports,politics,business,health'>
<input id='thresh' type='number' step='0.05' min='0' max='1' value='0.1'>
<button onclick='go()'>Classify</button>
<div id='out'></div>
<script>
async function go(){
  const t=document.getElementById('text').value;
  const l=document.getElementById('labels').value.split(',').map(s=>s.trim());
  const th=parseFloat(document.getElementById('thresh').value);
  const r=await fetch('/classify',{method:'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({text:t, labels:l, threshold:th})});
  const d=await r.json();
  let h='<h3>Results:</h3>';
  d.results.forEach(x=>{
    const pct=(x.score*100).toFixed(1);
    h+=`<div class='result'><b>${x.label}</b>: ${pct}%
      <div style='background:#ddd'><div class='bar'
style='width:${pct}%'></div></div></div>`;
  });
  document.getElementById('out').innerHTML=h;
}

```

```
</script></body></html>'''

@app.route('/')
def home(): return render_template_string(HTML)

@app.route('/classify', methods=['POST'])
def classify():
    d=request.json
    result=pipeline(d['text'],d['labels'],
                    threshold=d.get('threshold',0.1))
    scores={pred['label']:pred['score']
            for item in result for pred in item}
    try: wandb.log(scores)
    except: pass
    return jsonify({'results': [{'label':k,'score':v}
                                for k,v in scores.items()]})

if __name__=='__main__':
    print('Starting at http://localhost:8080')
    app.run(host='0.0.0.0', port=8080)
```

FILE: run_model.py v4 — Kaggle v1 (threshold=0.3)

Purpose: Classify 3263 disaster tweets; fixed threshold of 0.3

ISSUE: All 3263 tweets = disaster. Kaggle public score: 0.42966

ISSUE: Fixed threshold 0.3 too low — model scores nearly all tweets above 0.3 so all get label=1

```
# run_model.py Version 4: Kaggle submission v1 (FAILED)
# Kaggle score: 0.42966 | All 3263 predicted as disaster

import torch, wandb, pandas as pd
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3',
                 name='kaggle-disaster-tweets')
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                     torch_dtype=torch.float32)
model = model.float()
pipeline = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')
labels = ['disaster', 'not disaster']
predictions = []

for i, row in test_df.iterrows():
    text = str(row['text'])
    result = pipeline(text, labels, threshold=0.1)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_score = scores.get('disaster', 0)
    # BUG: 0.3 is too low — most scores exceed 0.3
    prediction = 1 if disaster_score > 0.3 else 0
    predictions.append(prediction)
    if i % 100 == 0:
        wandb.log({'progress':i, 'disaster_score':disaster_score})

submission = pd.DataFrame({'id':test_df['id'], 'target':predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
# Output: 1    3263  <- ALL disaster (wrong!)
run.finish()
```

FILE: run_model.py v5 — Kaggle v2 (label comparison)

Purpose: Compare disaster vs not-disaster scores to decide prediction

ISSUE: All 3263 tweets = not disaster. Score not improved

ISSUE: not_disaster label always scores higher than disaster label — model is biased toward the negative class

```
# run_model.py Version 5: Direct label comparison (FAILED)

import torch, wandb, pandas as pd
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3',
                 name='kaggle-disaster-tweets-v2')
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                     torch_dtype=torch.float32)
model = model.float()
pipeline = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')
labels = ['disaster', 'not disaster']
predictions = []

for i, row in test_df.iterrows():
    text = str(row['text'])
    result = pipeline(text, labels, threshold=0.1)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_score = scores.get('disaster', 0)
    not_disaster_score = scores.get('not disaster', 0)
    # BUG: not_disaster always wins in this model
    prediction = 1 if disaster_score > not_disaster_score else 0
    predictions.append(prediction)
    if i % 100 == 0:
        wandb.log({'progress':i, 'disaster_score':disaster_score})

submission = pd.DataFrame({'id':test_df['id'], 'target':predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
# Output: 0    3263  <- ALL not disaster (wrong!)
run.finish()
```

FILE: run_model.py v6 — FINAL Median Threshold (BEST)

Purpose: Collect all scores, use median as dynamic threshold — best Kaggle result

SUCCESS: Kaggle score 0.48605 | disaster=1631 not_disaster=1632 (balanced)

SUCCESS: KEY INSIGHT: Median threshold adapts to the model's score distribution → balanced 50/50 predictions

```
# run_model.py Version 6 FINAL (Kaggle score: 0.48605)
# docker run --rm -v model -v /data -v ~/.netrc gliclass-model

import torch, wandb, pandas as pd, statistics
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3',
                 name='kaggle-disaster-tweets-v3')

print('Loading model...')
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                     torch_dtype=torch.float32)
model = model.float()
pipeline = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label',
    device='cpu'
)

test_df = pd.read_csv('/data/test.csv')
train_df = pd.read_csv('/data/train.csv')
print(f'Test: {len(test_df)} tweets')
print(train_df['target'].value_counts())

# Step 1 — collect ALL disaster scores across full test set
labels = ['disaster', 'not disaster']
disaster_scores = []

for i, row in test_df.iterrows():
    text = str(row['text'])
    # threshold=0.01 ensures we get scores for every tweet
    result = pipeline(text, labels, threshold=0.01)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_scores.append(scores.get('disaster', 0))
    if i % 100 == 0:
        print(f' {i}/{len(test_df)} score={disaster_scores[-1]:.3f}')
        wandb.log({'progress': i,
                  'disaster_score': disaster_scores[-1]})

# Step 2 — median as dynamic threshold
median_score = statistics.median(disaster_scores)
print(f'Median score: {median_score:.4f}') # 0.4502

# Step 3 — predict: above median = disaster
predictions = [1 if s > median_score else 0
               for s in disaster_scores]

# Step 4 — save submission CSV
```



```
submission = pd.DataFrame({'id':test_df['id'],
                           'target':predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
# Output: 1->1631  0->1632  (balanced!)

wandb.log({'median_threshold': median_score,
          'total_disaster':    int(sum(predictions)),
          'total_not_disaster': int(len(predictions)
                                   - sum(predictions))})
run.finish()
print('Done! Kaggle score: 0.48605')
```

FILE: run_model.py v7 — Better Descriptive Labels

Purpose: Use longer, more descriptive labels to improve model context

SUCCESS: Submitted as v4-better-labels — results pending

```
# run_model.py Version 7: Descriptive labels for better accuracy

import torch, wandb, pandas as pd, statistics
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3',
                 name='kaggle-disaster-tweets-v4-better-labels')
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')

# Longer descriptive labels give more context to the model
labels = [
    'natural disaster emergency crisis fire flood earthquake',
    'normal everyday tweet unrelated to disaster'
]

disaster_scores = []
for i, row in test_df.iterrows():
    text = str(row['text'])
    result = pipeline(text, labels, threshold=0.01)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_scores.append(scores.get(labels[0], 0))
    if i % 100 == 0:
        wandb.log({'progress':i,
                  'disaster_score':disaster_scores[-1]})

median_score = statistics.median(disaster_scores)
predictions = [1 if s > median_score else 0
               for s in disaster_scores]
submission = pd.DataFrame({'id':test_df['id'],
                           'target':predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
wandb.log({'median_threshold': median_score,
          'total_disaster':    int(sum(predictions))})
run.finish()
```

Section 8: Challenges & Learnings

8.1 What Was Hard

Challenge	What Happened	How We Solved It
WSL venv on NTFS drive	python3 -m venv on F: drive gave 'Operation not permitted'	Created venv in WSL home (~/) — only project files on F: drive
HF_TOKEN overriding login	401 Unauthorized even after hf auth login — old env var took precedence	os.environ.pop('HF_TOKEN', None) before every login() call
huggingface-cli deprecated	Command not found — HF renamed CLI to 'hf'	Switched to: hf auth login and hf auth whoami
BFloat16 vs Float32 error	RuntimeError on CPU inference — model weights in BFloat16	Added model.float() after model loading
All tweets = disaster (v1)	Fixed threshold 0.3 too low for this model's score range	Switched to median-based dynamic threshold
All tweets = not disaster (v2)	not_disaster label always scored higher	Dropped direct label comparison; used median of disaster scores only
Docker credential error	docker: error getting credentials	rm ~/.docker/config.json && echo '{}' > ~/.docker/config.json
56 GB hf_cache locked by Docker	Docker Desktop stored vhdx files inside project's hf_cache folder	Stopped Docker Desktop, deleted via PowerShell Admin
VSCoDe not in WSL mode	F: drive paths invisible; kernel not found	Always open VSCoDe via 'code .' from WSL terminal
W&B key too short	API key rejected — must be 40+ chars	Generated new token from wandb.ai/authorize

8.2 What We Would Do Differently

- Fine-tune GLiClass on the disaster tweets training set rather than using zero-shot — would likely improve score from 0.48 to 0.65+
- Use GPU-enabled Docker base image (nvidia/cuda) to speed up inference from ~45 mins to ~2 mins for 3263 tweets
- Store model weights in a Docker volume or S3 bucket instead of local F: drive for reproducibility
- Set up a proper .env file with python-dotenv from the start rather than managing tokens manually
- Add a data validation step (Great Expectations or pandera) to assert tweet text is non-null before inference
- Push Docker image to Docker Hub with proper versioning tags (v1, v2, latest) from the start
- Set up the GitHub Actions workflow before starting development — would have caught environment issues earlier

8.3 Key Learnings

- WSL Fedora Remix behaves differently from Ubuntu — dnf vs apt, NTFS permission limitations
- Zero-shot classifiers are sensitive to label phrasing — longer, more descriptive labels can improve accuracy
- Median-based thresholding is a simple but effective way to produce balanced predictions from uncalibrated scores
- Docker volumes are the right way to handle large model files — never bake them into the image
- W&B is invaluable for tracking experiments — being able to compare runs visually saved significant debug time
- Always test credentials in isolation (`test_connections.py`) before running the full pipeline

Section 9: Final Summary — All Components with Links

Component	Status	Details	Public Link
WSL Fedora Remix	Done	Windows WSL Fedora distro	—
VSCode + Jupyter	Done	WSL mode, Python (MLOps-A3) kernel	—
Python venv	Done	~/mlops_venv, all packages installed	—
HuggingFace Login	Done	User: ausafahmad	huggingface.co/ausafahmad
HF Model	Done	gliclass-llama-1.3B downloaded	Model Card
Docker WSL	Done	Docker Desktop → FedoraRemix	Docker Hub
Docker Image	Done	gliclass-model:latest built & pushed	Pull Image
Flask Web UI	Done	Interactive UI at :8080	localhost:8080 (local)
Kaggle Auth	Done	ausafaitj, ~/.kaggle/kaggle.json	My Profile
Kaggle Dataset	Done	Disaster tweets, 3263 test rows	Competition
W&B Connected	Done	Project: mlops-assignment3	Dashboard
W&B Runs	Done	5 runs logged with metrics	Best Run
GitHub Repo	Done	Public, includes workflow files	Repository
GitHub Actions	Done	inference.yml workflow passing	Actions
Kaggle Submit v1	0.42966	threshold=0.3, all disaster	Submissions
Kaggle Submit v3	0.48605	Median threshold — BEST	Leaderboard